

Dimension Reduction

Author: Rohit Mande
Founder and CEO, intrrvu.ai

Website: <https://intrrvu.ai>



Index

- Why is dimension reduction needed?
- Principal Component Analysis ✓
- PCA Steps ✓
- Tips for applying PCA ✓

Website: <https://inttrvu.ai>

data with high dimensions:

- ① storage
- ② process
- ③ transfer
- ④ visualization
- ⑤ analyse
- ⑥ multicollinearity

Mo

Need of dimension reduction

Website: <https://inttrvu.ai>

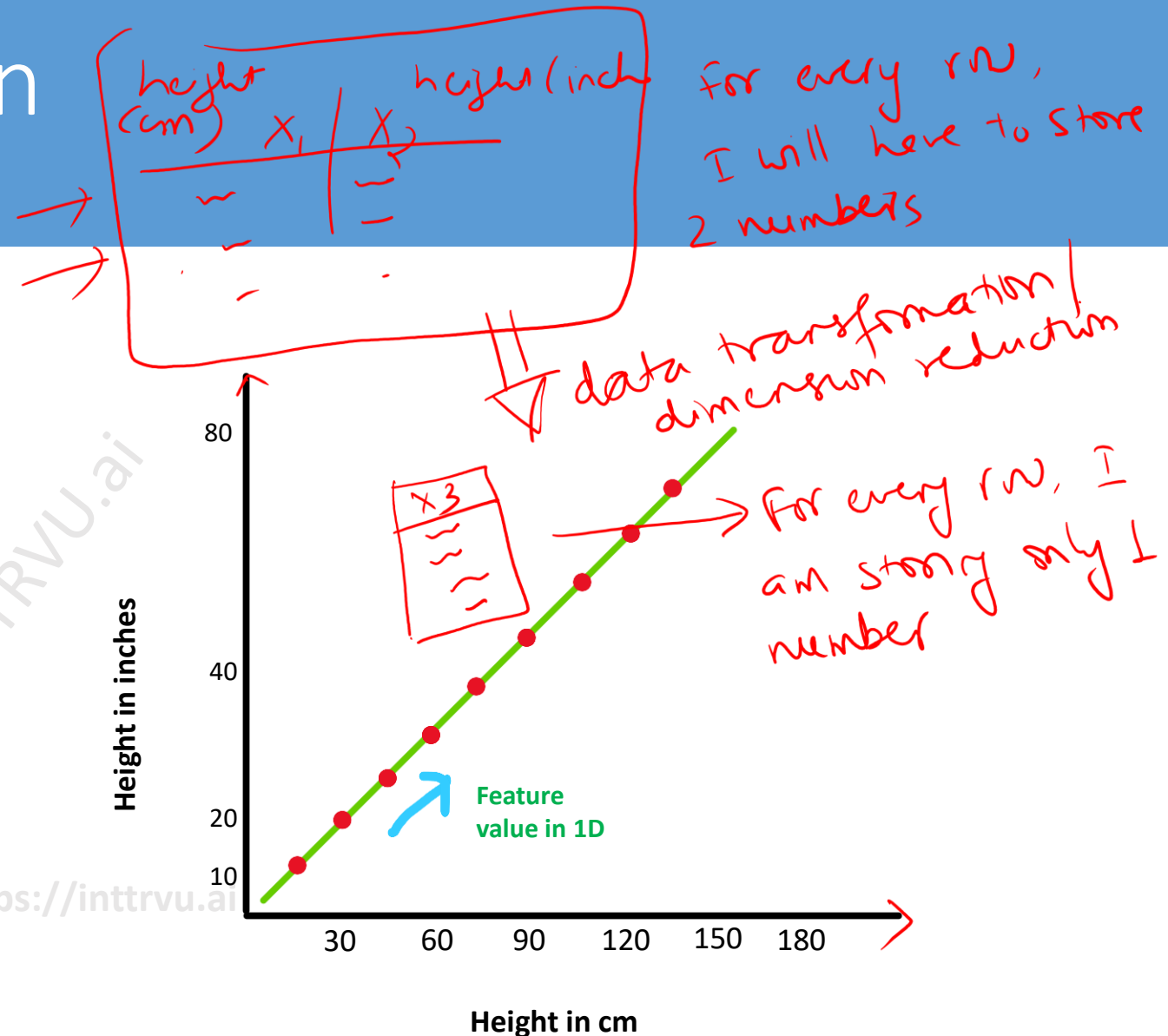
Why is dimension reduction needed?

- In real world problems we may get data with very high dimension which makes it difficult to store , visualize and use for ~~machine learning model~~ (E.g. similar outfit visualization, medical data visualization)
- Data might consist of features which provide similar information e.g. correlated features
Multicollinearity
- Data might have redundant features e.g. feature which provides height of person in inches and another feature which provides height in the cm

Dimension reduction

- X and Y axis same feature – height of person but measures in cms and inches respectively
- Both the features are not needed, and we can use any one of them
- Using dimension reduction both the features can be converted from 2D to 1D by taking location on green line as the feature value

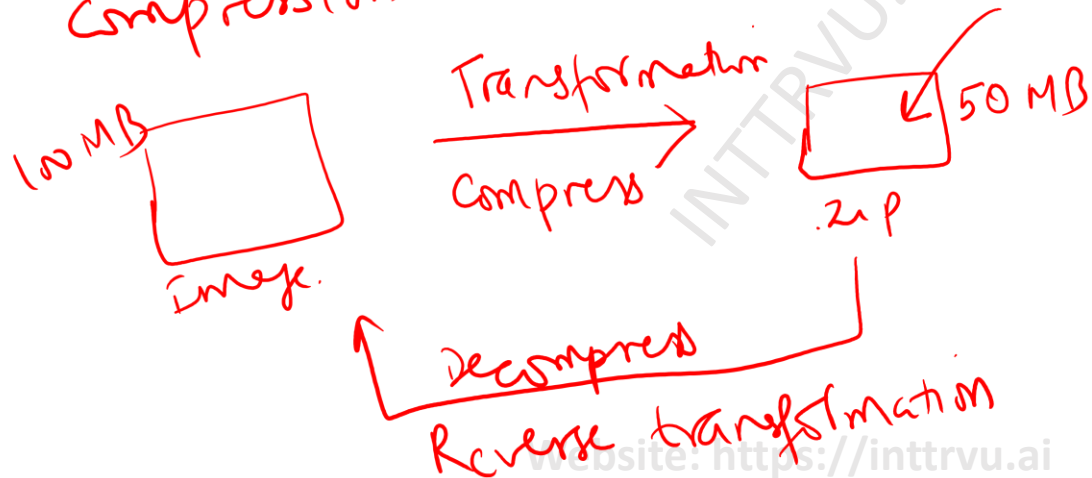
In real world datasets presence of such redundant features in dataset might not be easy to spot



Examples

① Images 100×100 pixels $\Rightarrow 10000$ features $\Rightarrow$ high dimensional
Storage, processing, ML classification
dimensionality reduction
For every image instead of storing 10000; just store 50-100 features

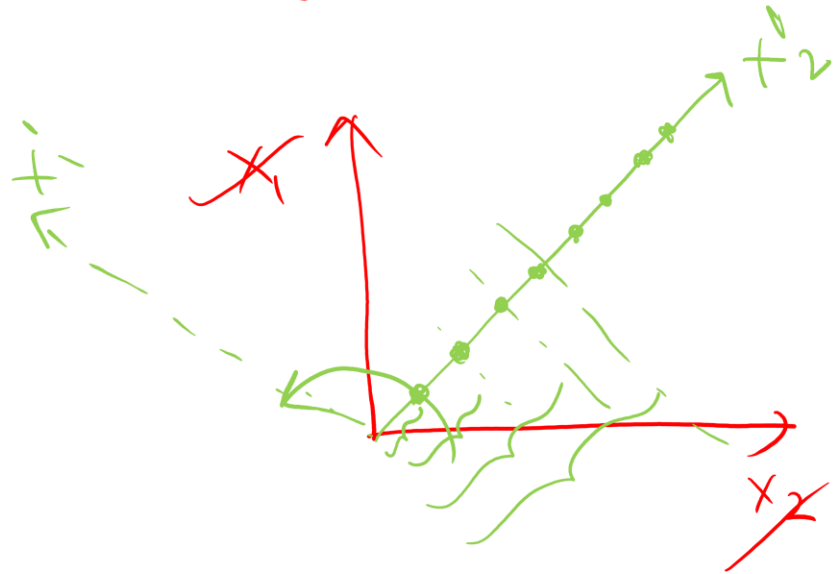
② File Compression



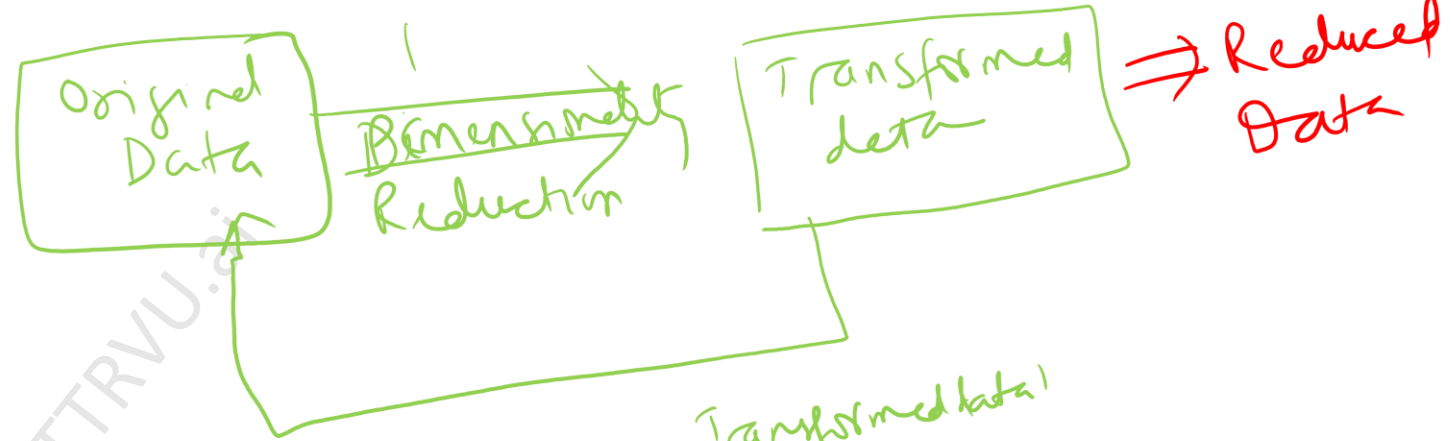
Loss of information

$\Rightarrow$ Tolerable loss of information

I want an algo which automatically looks at the features and keep only the required columns.



height in cm
height in inches } $\Rightarrow \begin{matrix} x_2' \\ x_1' \end{matrix}$



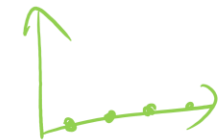
We linearly transform our data

Original Data
Row 1 (x_1, x_2)



Transformed data
 (x_1', x_2')

Website: <https://inttrvu.ai>



Dimension Reduction

for visualization

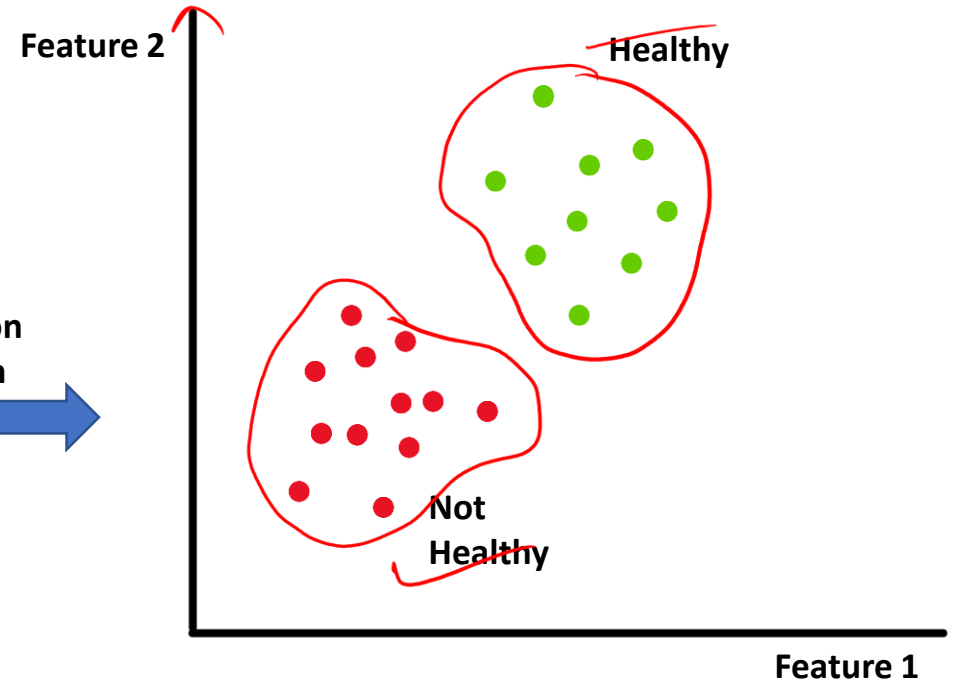
always keep top 2 features (which are most important)

Input data consists of multiple features

Dimension reduction is needed to visualize data in 2D

Persona Id	Height (cm) ✓	Height (inch) ✓	weight	BMI	Daily walking (km)
1	150	60	60	26.70	2.5
2	175	70	65	21.22	3.0
3	200	80	90	27.78	2.2

Dimension reduction



↓ data transformation

f ₁	f ₂	f ₃	f ₄	f ₅

⇒ data reduction f₁ & f₂

PCA

unsupervised algorithm

Principal Component Analysis

Website: <https://inttrvu.ai>

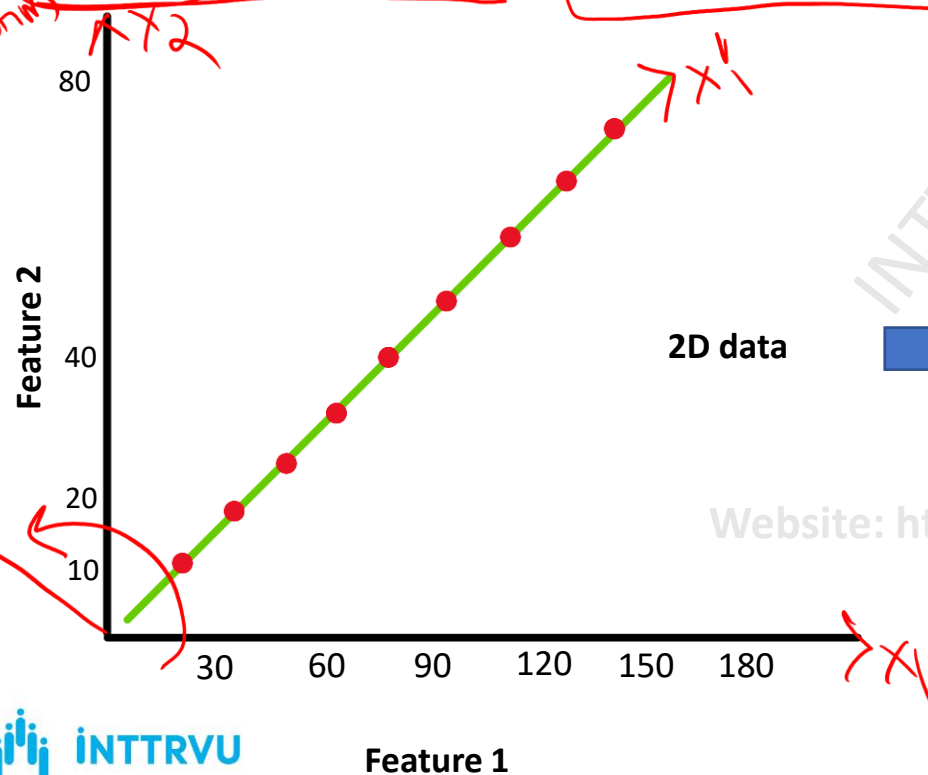
Rotation of axes is a linear transformation

Principal Component Analysis

- Principal Component Analysis (PCA) is unsupervised learning method for dimension reduction
- PCA reduces dimension by linearly transforming the data into a new coordinate system where (most of) the variation in the data can be described with fewer dimensions than the initial data.

loss of knowledge.

knowledge or information



1D data in new coordinate system

Original $(x_1, x_2) \rightarrow$ Transformed (x_1', x_2') $\Rightarrow$ Reduced (x_1')

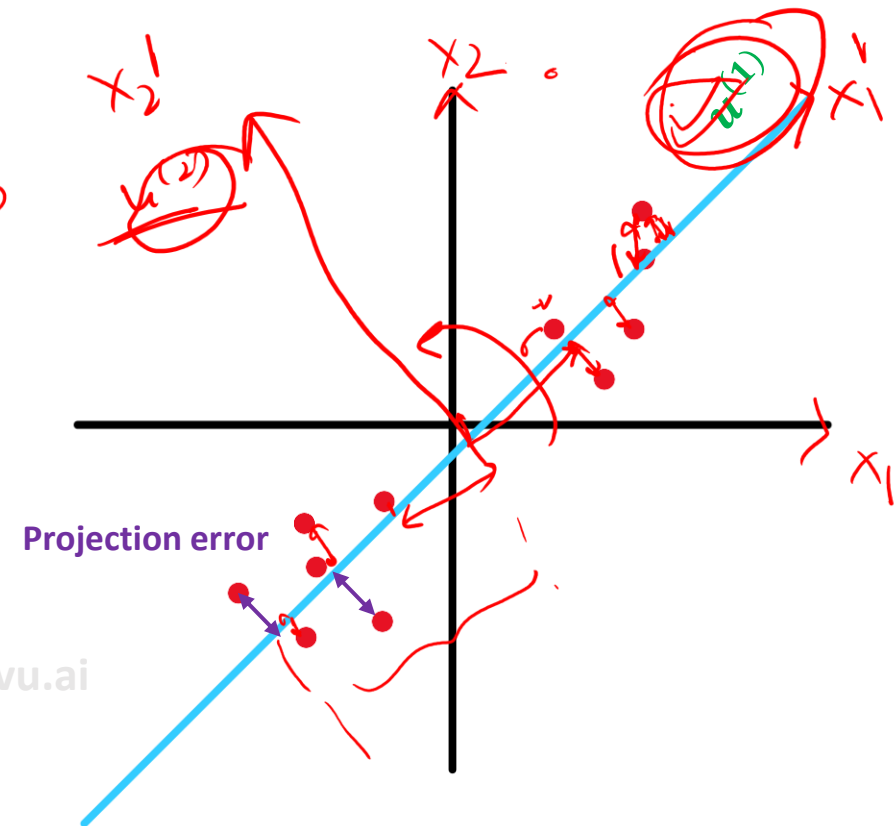
PCA

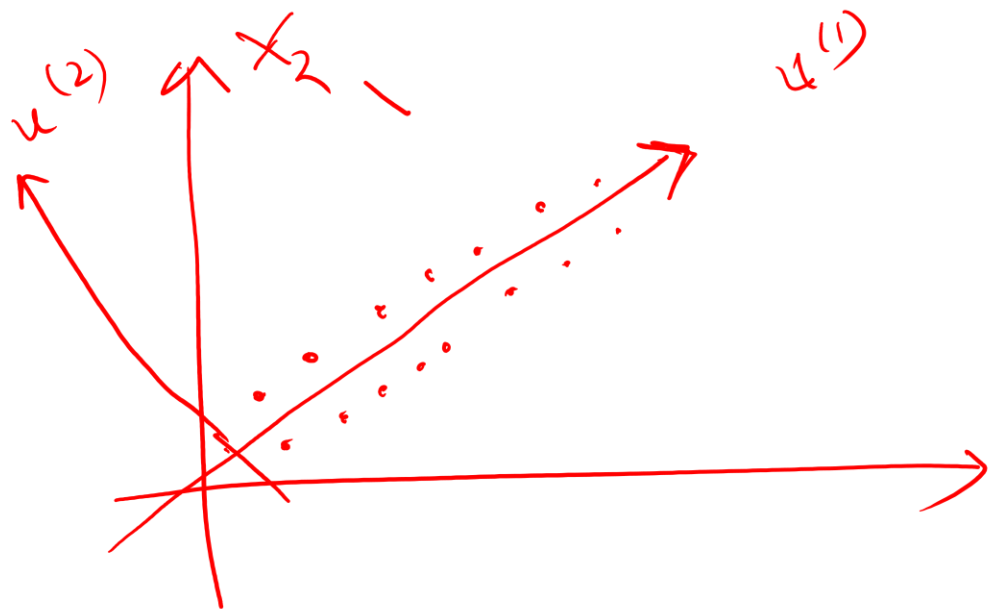
Goal is to find $u^{(1)}$ and $u^{(2)}$
 $(x_1, x_2) \rightarrow (x_1', x_2') \rightsquigarrow x_1'$
by letting go of x_2'

- From 2-dimensional to 1-dimensional : Find the direction (vector $u^{(1)}$) on which data can be projected to minimize projection error
- From N-dimensional to K-dimensional : Find the directions (k vectors $u^{(1)}, u^{(2)}, \dots, u^{(k)}$) on which data can be projected to minimize projection error

PCA is Not a linear regression

$u^{(1)}$ and $u^{(2)}$ are new coordinate axes





$\Rightarrow$



$\Downarrow$ Data Reduction

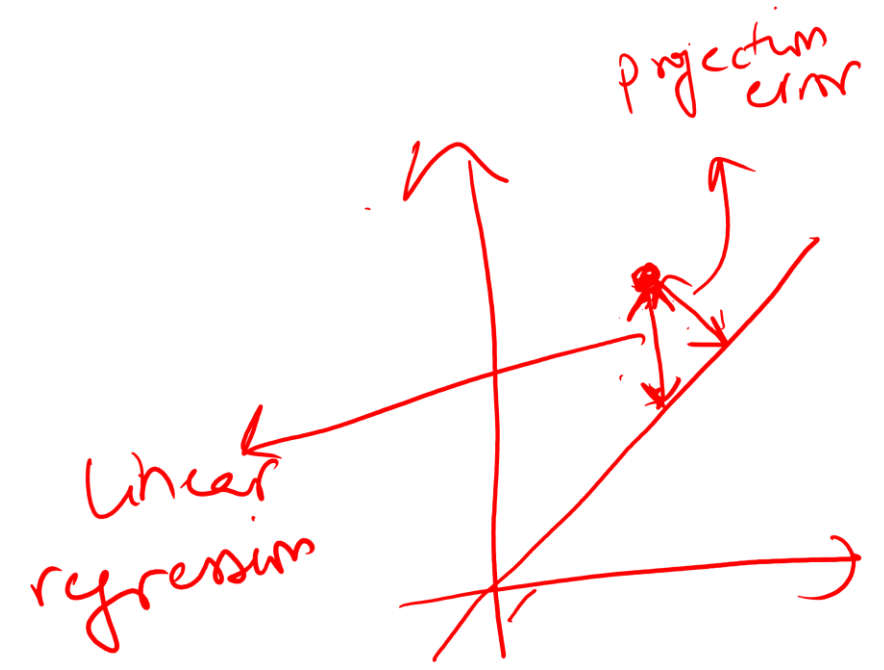
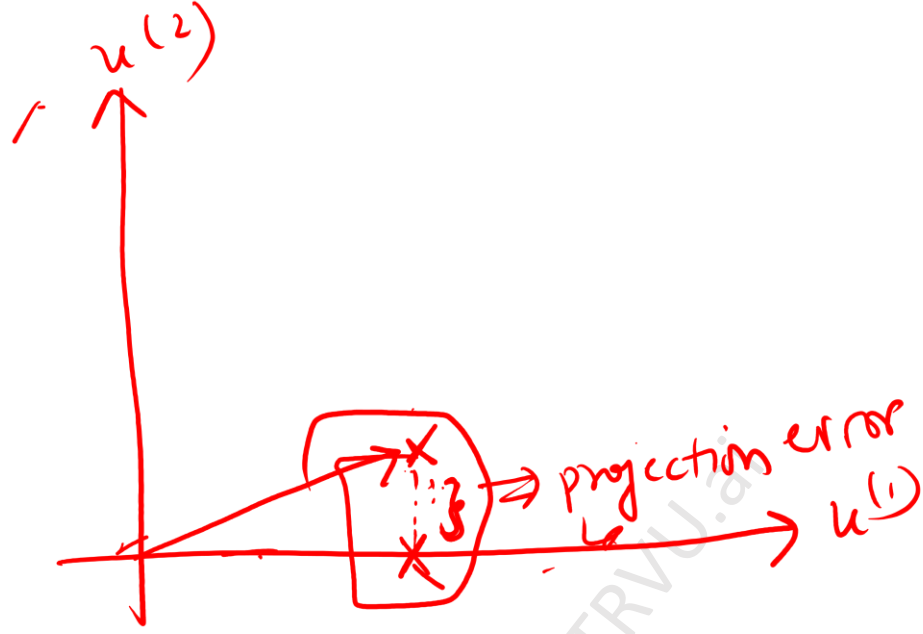


$$u^{(1)}, u^{(2)} = ??$$

$$(x_1, x_2) \Rightarrow (u^{(1)}, u^{(2)})$$

$$(x_1, x_2, x_3) \Rightarrow (u^{(1)}, u^{(2)}, u^{(3)}) \Rightarrow u^{(1)}, u^{(2)}$$

Website: <https://inttrvu.ai>



Website: <https://inttrvu.ai>

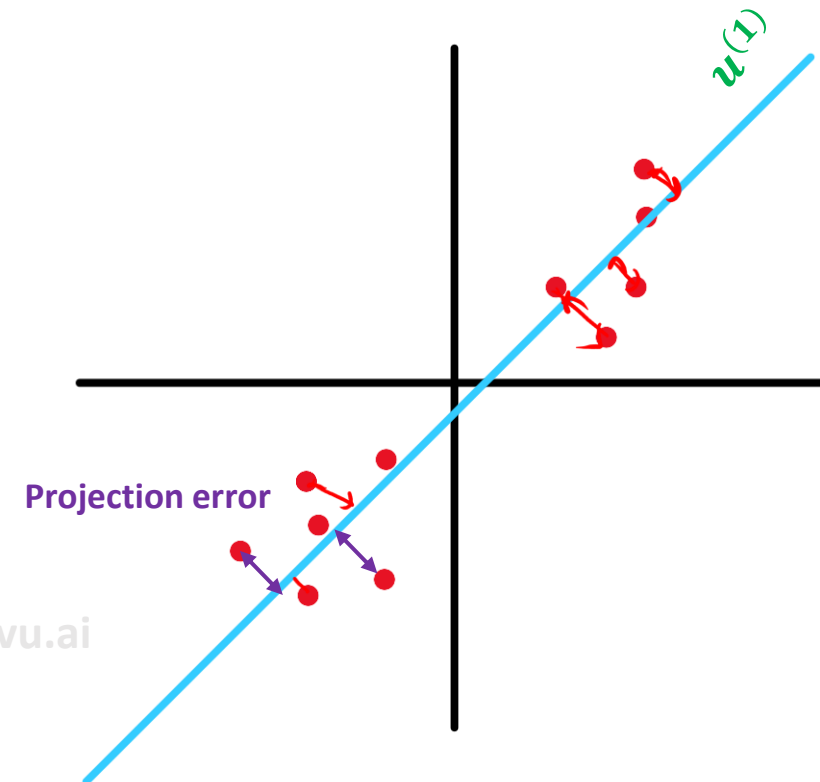
PCA

N dimension $\rightarrow$ N dimension
Data Transformation

$\rightarrow$ K dimension
Data Reduction

From N -dimensional to K -dimensional : Find the directions (k vectors $u^{(1)}, u^{(2)}, \dots, u^{(k)}$) on which data can be projected such that:

- Projection error is minimized
- New features are created by projecting to new-coordinate system using linear transformation
- All the vectors are orthogonal (perpendicular) each other
- All the features created are non-correlated

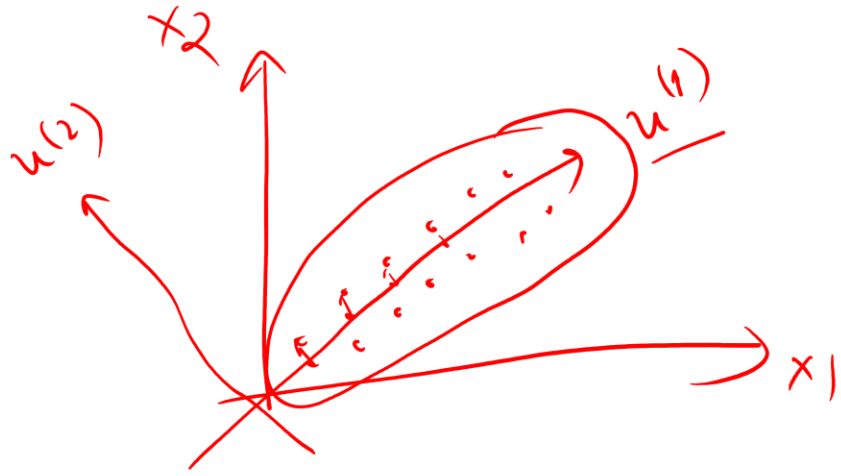


website: <https://inttrvu.ai>



$u^{(1)}$ & $u^{(2)}$ $\Rightarrow$ orthogonal to each other
 $\Rightarrow u^{(1)}$ and $u^{(2)}$ are not correlated
 $\Rightarrow$ we have eliminated the problem of multicollinearity
 $\Rightarrow$ these features are independent of each other

Website: <https://inttrvu.ai>



$u^{(1)}$ —
 $u^{(2)}$

$u^{(1)}$ is a linear combination of x_1 & x_2
 $u^{(2)}$ is also a linear combination of x_1 & x_2

INTTRVU.ai

Website: <https://inttrvu.ai>

PCA Algorithm steps

① Data preprocessing

say you have M features

x_1	x_2	x_3	x_4	$\dots$	x_M
μ_1	μ_2	μ_3	μ_4		μ_M

$X_1 - \mu_1$	$X_2 - \mu_2$	$X_3 - \mu_3$	...	$X_M - \mu_M$
$1 - \mu_1$	$5 - \mu_2$			
$2 - \mu_1$	$100 - \mu_2$			
$N - \mu_1$	$7 - \mu_2$			

Website: [http://www.intel.com](#)

Hand-drawn diagram illustrating a data matrix X with dimensions $N \times M$.

The matrix is represented as a grid with columns labeled $x_1, x_2, x_3, x_4, \dots, x_M$ and rows labeled $1, 2, \dots, N$.

Below the columns, the labels $\mu_1, \mu_2, \mu_3, \mu_4, \dots, \mu_M$ are shown, with the word "mean" written below μ_1 .

PCA Steps

Website: <https://inttrvu.ai>

PCA Steps

1) Data pre-processing:

If training set consists of n features $x^{(1)}, x^{(2)}, \dots, x^{(n)}$

Calculate mean for every feature

$$\mu_i = \frac{1}{m} \sum_{j=1}^m x_j^{(i)}$$

- Substitute each feature value $x_j^{(i)}$ by $(x_j^{(i)} - \mu_i)$ (mean normalization)
- If features are different scale – scale the features so that all the features are in similar range

PCA Steps

$$(X - \mu) \cdot (X - \mu)^T = \text{covariance matrix}$$

$x_1 \quad x_2 \quad \dots \quad x_n$

$\begin{bmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \dots & \text{Cov}(x_1, x_n) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \dots & \text{Cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \dots & \dots & \text{Var}(x_n) \end{bmatrix}$

2) Calculate covariance matrix:

Covariance is a measure of the relationship between two random variables and to what extent, they change together

Covariance is a measure of the relationship between two random variables and to what extent, they change

$$\Sigma = \frac{1}{n-1} ((X - \mu) \cdot (X - \mu)^T)$$

n is number of features

X is matrix of input feature vectors (original data)

μ is matrix of mean values of features (original data)

$$\begin{bmatrix} \text{Var}(x_1) & \dots & \text{Cov}(x_1, x_n) \\ \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \dots & \text{Var}(x_n) \end{bmatrix}$$

Step 3 Covariance Matrix Calculation

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

$\rho(X, Y)$ = Pearson correlation coefficient =

$$\frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

$$\begin{matrix} & X_1 & X_2 & X_j & \dots & X_M \\ \begin{matrix} X_1 \\ X_2 \\ \vdots \\ X_i \\ \vdots \\ X_M \end{matrix} & \begin{bmatrix} \sim & & & & \\ & \sim & & & \\ & & \vdots & & \\ & & & * & \\ & & & & \sim \end{bmatrix} \end{matrix}$$

Covariance matrix

$$\begin{aligned} & \text{Cov}(X_i, X_j) \\ &= E[(X_i - \mu_{X_i})(X_j - \mu_{X_j})] \end{aligned}$$

Website: <https://inttrvu.ai>

$$(X - \mu) = \begin{bmatrix} x_1 - \mu_1 & x_2 - \mu_2 & x_3 - \mu_3 & \dots & x_M - \mu_M \end{bmatrix}$$

$N \times M$

N

$$(X - \mu)^T (X - \mu) = \underline{M \times M}$$

$M \times M$ matrix

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_M \end{bmatrix}$$

Website: <https://inttrvu.ai>

INTTRVU.ai

Website: <https://inttrvu.ai>

$$\text{Cov}(X_i, X_j)$$

$$(X_i - \mu_i)^T \begin{bmatrix} a_1 & a_2 & \dots & a_n \end{bmatrix} \begin{bmatrix} (X_j - \mu_j) \\ b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$(X - \mu)^T$ $(X - \mu)$

$$\text{Cov}(X_i, X_j) = \frac{a_1 b_1 + a_2 b_2 + \dots + a_n b_n}{n-1} = E((X_i - \mu_i)(X_j - \mu_j))$$

Website: <https://inttrvu.ai>

PCA Steps

$$Ax = \lambda x$$



Covariance

matrix

$A \Rightarrow$ matrix x for which we want to calculate eigen value and vectors

$x \Rightarrow$ eigen vector
 $\lambda =$ eigen value

3) Eigen value decomposition

- Calculate Eigen values and Eigen vectors of the covariance matrix
- Eigen vectors represent orthogonal vectors on which data can be projected
- Eigen values represent amount of variance explained by the corresponding eigen vector

$$\% \text{ of info in } u^{(k)} = \lambda^{(k)} / \sum \lambda$$

Variance explained by each vector k = Eigen value of vector k / sum of all eigen values

these eigen vectors are $u^{(1)}, u^{(2)}, \dots, u^{(M)}$
 $\lambda^{(1)}, \lambda^{(2)}, \dots, \lambda^{(M)}$

Website: <https://inttrvu.ai>

$\lambda^{(i)}$ tells about the amount of information present in $u^{(i)}$

PCA Steps

4) Select k (number of component /dimension of new data)

- Select K which explains at least 90% variance of the data
- Higher the explained variance – lower the projection error

E.g. If PCA is applied to data with 100 features and sum of first 40 eigen values gives more than 90% variance then we can select first 40 eigen vectors (dimensions)

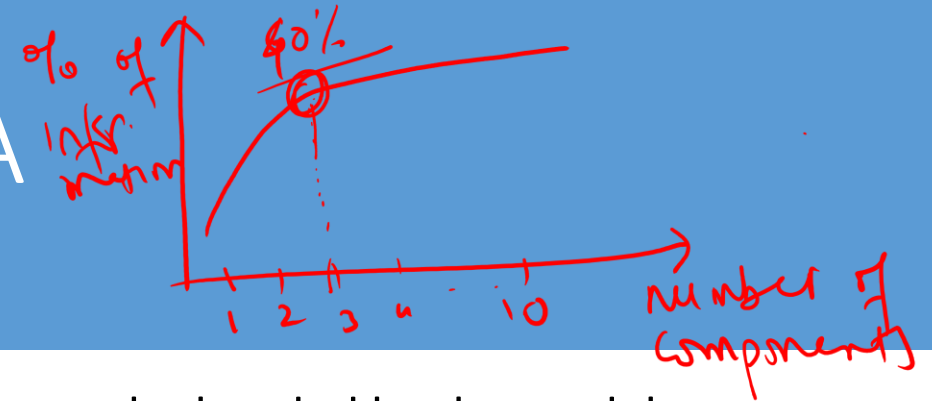
$$\begin{array}{l} u^{(1)} \\ u^{(2)} \\ u^{(3)} \\ \vdots \\ u^{(n)} \end{array} \quad \left. \begin{array}{l} \lambda^{(1)} \rightarrow a_1 / 50\% \\ \lambda^{(2)} \rightarrow a_2 / 10\% \\ \lambda^{(3)} \rightarrow a_3 / 9\% \\ \vdots \\ \lambda^{(n)} \end{array} \right\} \lambda^{(1)} > \lambda^{(2)} > \lambda^{(3)} \dots > \lambda^{(n)}$$

Keep those many top $u^{(i)}$, which will give you 90% of information

Tips for applying PCA

Website: <https://inttrvu.ai>

Useful tips for applying PCA



- Never use PCA just to reduce the dimension if original features can be handled by the model
- Use of PCA reduces model explainability as we don't know which features have contributed to the creation of each principal component
- Select number of components based on explained variance and number of features which model can handle – never select using elbow curve
If you are using PCA for visualization then use $u^{(1)}$ and $u^{(2)}$
 $\lambda^{(i)}, u^{(i)}$
- Apply PCA only on training data and use the learned transformations on testing and prediction data. Never apply PCA on whole data and then do train/test split

Use of PCA

Without PCA

Supervised Learning / Prediction

①

Original Data

- High dimensional
- Multicollinearity

Training Data

Testing Data

Live Data



Model

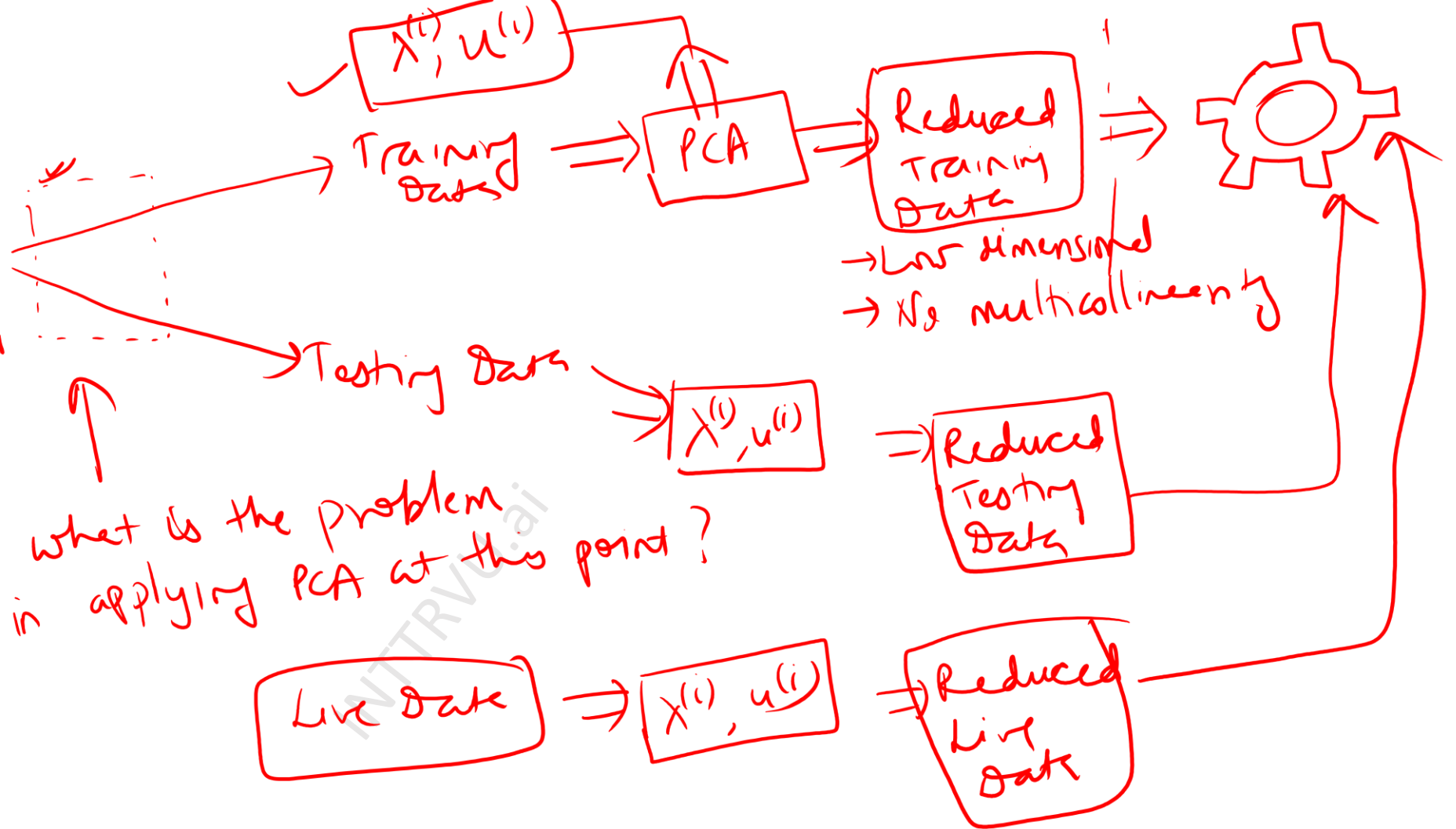
Final Testing Metrics

Website: <https://inttrvu.ai>

Use the PCA

Original Data

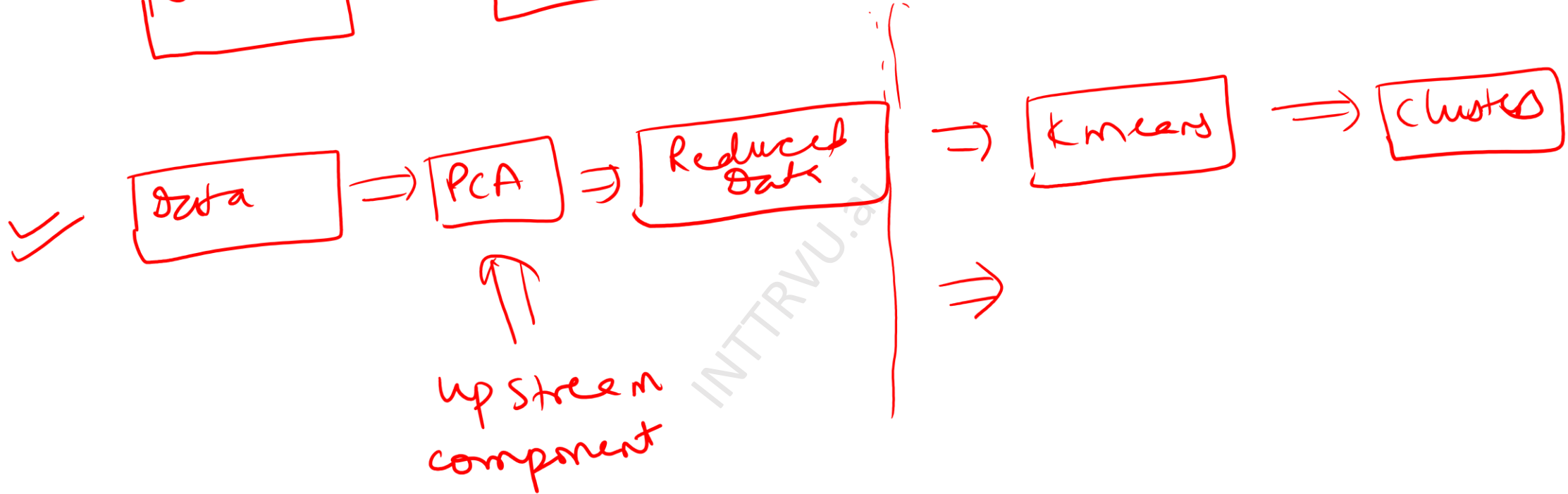
- ① High Dimensional
- ② Multi collinear



What is the problem in applying PCA at this point?

Website: <https://inttrvu.ai>

② Use PCA for clustering



Website: <https://inttrvu.ai>

Questions?

Website: <https://inttrvu.ai>